Churn Model Comparison Demo

Bryan Keating

2026-11-08

Table of contents

1 IBM Watson Customer Churn Data Set 1

2 Report Outcome 2

3 Data Cleaning 3

4 Data Inspection 4

5 Train-test Split 5

6 Model Creation 6

7 Model Testing 7

8 Accuracy Discussion 8
- 8.1 Claude . . . 8
- 8.2 Athlera . . . 9

9 Performance Comparison Metrics 10

1 IBM Watson Customer Churn Data Set

The `wu_churn` data set is a collection of IBM Watson customer data collected in September 2018. The data contains a factor variable of whether or not the customer has churned, and other related information about the customer.

```r
# Install and load the dataset
# install.packages("modeldata")
remove(list = ls())
library(modeldata)
library(tidyverse)
data(wa_churn)
str(wa_churn)
```

```
tibble [7,043 x 20] (S3: tbl_df/tbl/data.frame)
 $ churn            : Factor w/ 2 levels "Yes","No": 2 2 1 2 1 1 2 2 1 2 ...
 $ female           : num [1:7043] 1 0 0 0 1 1 0 1 1 0 ...
 $ senior_citizen   : int [1:7043] 0 0 0 0 0 0 0 0 0 0 ...
 $ partner          : num [1:7043] 1 0 0 0 0 0 0 0 1 0 ...
 $ dependents       : num [1:7043] 0 0 0 0 0 0 1 0 0 1 ...
 $ tenure           : int [1:7043] 1 34 2 45 2 8 22 10 28 62 ...
 $ phone_service    : num [1:7043] 0 1 1 0 1 1 1 0 1 1 ...
 $ multiple_lines   : Factor w/ 3 levels "No","No phone service",..: 2 1 1 2 1 3 3 2 3 1 ...
 $ internet_service : Factor w/ 3 levels "DSL","Fiber optic",..: 1 1 1 1 2 2 2 1 2 1 ...
 $ online_security  : Factor w/ 3 levels "No","No internet service",..: 1 3 3 3 1 1 1 3 1 3
 $ online_backup    : Factor w/ 3 levels "No","No internet service",..: 3 1 3 1 1 1 3 1 1 3
 $ device_protection: Factor w/ 3 levels "No","No internet service",..: 1 3 1 3 1 3 1 1 3 1
 $ tech_support     : Factor w/ 3 levels "No","No internet service",..: 1 1 1 3 1 1 1 1 3 1
 $ streaming_tv     : Factor w/ 3 levels "No","No internet service",..: 1 1 1 1 1 3 3 1 3 1
 $ streaming_movies : Factor w/ 3 levels "No","No internet service",..: 1 1 1 1 1 3 1 1 3 1
 $ contract         : Factor w/ 3 levels "Month-to-month",..: 1 2 1 2 1 1 1 1 1 2 ...
 $ paperless_billing: num [1:7043] 1 0 1 0 1 1 1 0 1 0 ...
 $ payment_method   : Factor w/ 4 levels "Bank transfer (automatic)",..: 3 4 4 1 3 3 2 4 3 1
 $ monthly_charges  : num [1:7043] 29.9 57 53.9 42.3 70.7 ...
 $ total_charges    : num [1:7043] 29.9 1889.5 108.2 1840.8 151.7 ...
```

2 Report Outcome

The goal of this report is to use the `wu_churn` data set to compare the performance between two models trained to predict whether a customer will churn based on their observed characteristics. Ultimately, this will show quantitative methods of scoring model performance and derive metrics for proving the strongest predictor choice.

3 Data Cleaning

Convert numeric categorical columns to factor

```
# Backup dataset
backup_wa_churn <- wa_churn
factor_cols <- c('female','senior_citizen','partner',
                 'dependents','phone_service','paperless_billing')

# Factoring
wa_churn[factor_cols] <- lapply(
  wa_churn[factor_cols],
  function(x) {factor(x, levels = c(0,1), labels = c('No','Yes'))})

wa_churn$churn <- factor(wa_churn$churn, levels = c('No','Yes'))

# Comparison check
head(wa_churn,3)
```

```
# A tibble: 3 x 20
  churn female senior_citizen partner dependents tenure phone_service
  <fct> <fct>  <fct>          <fct>   <fct>       <int> <fct>
1 No    Yes    No             Yes     No              1 No
2 No    No     No             No      No             34 Yes
3 Yes   No     No             No      No              2 Yes
# i 13 more variables: multiple_lines <fct>, internet_service <fct>,
#   online_security <fct>, online_backup <fct>, device_protection <fct>,
#   tech_support <fct>, streaming_tv <fct>, streaming_movies <fct>,
#   contract <fct>, paperless_billing <fct>, payment_method <fct>,
#   monthly_charges <dbl>, total_charges <dbl>
```

```
head(backup_wa_churn,3)
```

```
# A tibble: 3 x 20
  churn female senior_citizen partner dependents tenure phone_service
  <fct>  <dbl>          <int>   <dbl>      <dbl>  <int>         <dbl>
1 No         1              0       1          0      1             0
2 No         0              0       0          0     34             1
3 Yes        0              0       0          0      2             1
# i 13 more variables: multiple_lines <fct>, internet_service <fct>,
#   online_security <fct>, online_backup <fct>, device_protection <fct>,
```

```
#   tech_support <fct>, streaming_tv <fct>, streaming_movies <fct>,
#   contract <fct>, paperless_billing <dbl>, payment_method <fct>,
#   monthly_charges <dbl>, total_charges <dbl>
```

Inspect multicollinearity in the continuous subset. As expected, total charges is highly correlated with monthly charges and tenure and can be dropped to avoid model redundancy.

```
# Continous subset
continuous_cols <- c('tenure','monthly_charges','total_charges')

# Pearson correlation matrix
cor(na.omit(wa_churn[continuous_cols]))
```

```
                  tenure monthly_charges total_charges
tenure          1.0000000       0.2468618     0.8258805
monthly_charges 0.2468618       1.0000000     0.6510648
total_charges   0.8258805       0.6510648     1.0000000
```

```
# Drop redundant total_charges
wa_churn <- wa_churn |>
  select(-total_charges)
```

4 Data Inspection

Understanding the value possibilities for each variable and finding relationships between them will be helpful in determining which should be included in the model. Internet amenities are influenced by the existence of internet service. This will have to be considered when designing the model.

```
# Columns
unique_cols <- c('multiple_lines', 'internet_service', 'online_security',
                 'online_backup', 'device_protection', 'tech_support',
                 'streaming_tv', 'streaming_movies', 'contract')

# Unique value analysis
unique_list <- as.data.frame(lapply(wa_churn[unique_cols], unique))
unique_list
```

```
    multiple_lines internet_service     online_security       online_backup
1 No phone service              DSL                 No                 Yes
2               No      Fiber optic                Yes                  No
3              Yes               No No internet service No internet service
    device_protection        tech_support        streaming_tv
1                  No                  No                  No
2                 Yes                 Yes                 Yes
3 No internet service No internet service No internet service
     streaming_movies       contract
1                  No Month-to-month
2                 Yes       One year
3 No internet service       Two year
```

Getting a feel for total and categorical NA counts can identify columns that may need to be ommitted due to inadequate coverage. Since there are no NA values, there is no need to impute or interpolate.

```
total_na <- sum(is.na(wa_churn))
total_na
```

```
[1] 0
```

5 Train-test Split

The data set will be split randomly into training and testing sets. The models will be trained blind to avoid over-fitting issues. Their performances will then be tested against the test set.

A stratified random split method will be used to guarantee that the train and test sets have equal proportions of churned-to-not-churned customers.

```
# install.packages("rsample")
library(rsample)

# Reproducibility
set.seed(100)

# Create the split object, stratified by churn value
churn_split <- initial_split(wa_churn, prop = 0.75, strata = churn)

# Extract train and test sets
train_data <- training(churn_split)
```

```r
test_data <- testing(churn_split)

# Verify dimensions and churn proportions
nrow(train_data)
```

```
[1] 5281
```

```r
nrow(test_data)
```

```
[1] 1762
```

```r
prop.table(table(train_data$churn))
```

```
       No       Yes
0.7347093 0.2652907
```

```r
prop.table(table(test_data$churn))
```

```
       No       Yes
0.7343927 0.2656073
```

6 Model Creation

Two models will be created for this report:

- The `claude_model` will represent the prediction capabilities of a general LLM. It will be trained on two major predictor variables: `tenure` and `paperless_billing`
- The `athlera_model` will represent the prediction capabilities of Athlera's proprietary deterministic, guard-railed LLM. It will be trained on the following predictor variables:
  - `female` - gender of customer
  - `senior_citizen` - age bracket
  - `tenure` - length of subscription
  - `phone_service` - whether they are subscribed to a phone service

- `contract` - billing frequency
- `paperless_billing` - electronic or mailed billing
- `payment_method` - method of payment
- `monthly_charges` - monthly payment for services

The purpose of intentionally creating different strength models is to illustrate performance metrics that can be used for model comparison.

```
# Claude representation
claude_model <- glm(
  churn ~ tenure + paperless_billing,
  data = train_data,
  family = binomial(link = "logit"))

# summary(claude_model)

# Athlera representation
athlera_model <- glm(
  churn ~ female + senior_citizen + tenure + phone_service +
    contract + paperless_billing + payment_method + monthly_charges,
  data = wa_churn,
  family = binomial(link = "logit"))

# as.data.frame(coefficients(athlera_model))
```

7 Model Testing

Now that the models have been trained, it is time to see how accurately they predict the churn values in the test set.

```
# Testing claude_model
claude_probs <- predict(claude_model, newdata = test_data, type = "response")
claude_preds <- factor(
  ifelse(claude_probs > 0.5, "Yes", "No"),
  levels = c("No", "Yes"))

# Claude confusion matrix
claude_cm <- table(Actual = test_data$churn, Predicted = claude_preds)
claude_cm
```

```
      Predicted
Actual   No  Yes
   No  1169  125
   Yes  295  173
```

```
# Testing
athlera_probs <- predict(athlera_model, newdata = test_data, type = "response")
athlera_preds <- factor(
  ifelse(athlera_probs > 0.5, "Yes", "No"),
  levels = c("No","Yes"))

# Athlera confusion matrix
athlera_cm <- table(Actual = test_data$churn, Predicted = athlera_preds)
athlera_cm
```

```
      Predicted
Actual   No  Yes
   No  1167  127
   Yes  219  249
```

8 Accuracy Discussion

Since the majority of our data set (~73.5%) is non-churner customers, a model that predicts "No" every time will still have a 73.5% accuracy. Due to this, it is a stronger indicator of performance to compare how likely the models predicted a churn given the customer churned in reality. In other words, it is more fruitful to compare false negatives and true positives generated by the models.

8.1 Claude

```
# False negatives
claude_false_neg <- claude_cm["Yes","No"] / sum(claude_cm["Yes", ])
round(claude_false_neg, 4)
```

```
[1] 0.6303
```

```r
# True positives
claude_true_pos <- claude_cm["Yes", "Yes"] / sum(claude_cm["Yes", ])
round(claude_true_pos, 4)
```

```
[1] 0.3697
```

```r
# False positives
claude_false_pos <- claude_cm["No","Yes"] / sum(claude_cm["No", ])
round(claude_false_pos, 4)
```

```
[1] 0.0966
```

```r
# True negatives
claude_true_neg <- claude_cm["No","No"] / sum(claude_cm["No", ])
round(claude_true_neg, 4)
```

```
[1] 0.9034
```

8.2 Athlera

```r
# False negatives
athlera_false_neg <- athlera_cm["Yes","No"] / sum(athlera_cm["Yes", ])
round(athlera_false_neg, 4)
```

```
[1] 0.4679
```

```r
# True positives
athlera_true_pos <- athlera_cm["Yes", "Yes"] / sum(athlera_cm["Yes", ])
round(athlera_true_pos, 4)
```

```
[1] 0.5321
```

```r
# False positives
athlera_false_pos <- athlera_cm["No","Yes"] / sum(athlera_cm["No", ])
round(athlera_false_pos, 4)
```

```
[1] 0.0981
```

```r
# True negatives
athlera_true_neg <- athlera_cm["No","No"] / sum(athlera_cm["No", ])
round(athlera_true_neg, 4)
```

```
[1] 0.9019
```

9 Performance Comparison Metrics

Compiling the data

```r
library(scales)
library(ggplot2)

# Build the comparison dataframe
performance_data <- data.frame(
  Metric = rep(c("True Positives", "True Negatives", "False Positives",
                 "False Negatives"), 2),
  Model = rep(c("Claude", "Athlera"), each = 4),
  Accuracy = c(
    claude_true_pos, claude_true_neg, claude_false_pos, claude_false_neg,
    athlera_true_pos, athlera_true_neg, athlera_false_pos, athlera_false_neg))
performance_data$Metric <- factor(
  performance_data$Metric, levels = c("True Positives", "True Negatives",
                                      "False Positives", "False Negatives"))

ggplot(performance_data, aes(x = Metric, y = Accuracy, fill = Model)) +
  geom_col(position = "dodge")
```



Styled plot

```
# 2. Build the grouped bar chart
ggplot(performance_data, aes(x = Metric, y = Accuracy, fill = Model)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7) +
  geom_text(
    aes(label = percent(Accuracy, accuracy = 0.1)),
    position = position_dodge(width = 0.8),
    vjust = -0.5,
    size = 3.5
  ) +
  scale_y_continuous(
    labels = percent_format(),
    limits = c(0, 1.05),
    expand = expansion(mult = c(0, 0.05))
  ) +
  scale_fill_manual(values = c("Athlera" = "#2b5c8f", "Claude" = "#d97736")) +
  labs(
    title = "Model Performance Comparison",
    x = "Confusion Metric",
    y = "Rate (Proportion)") +
  theme_minimal(base_size = 13) +
  theme(
```

```
    legend.position = "top",
    panel.grid.major.x = element_blank(),
    plot.title = element_text(face = "bold", hjust = 0.5)
  )
```

